

Final Submission Notebook

This single notebook consolidates all three project parts (search, optimization, and learning) with runnable code, test cases, and recorded results.

Key data notes:

- Parts 1 and 2 use demo datasets because the official assignment datasets are not tracked in this repo.
- Part 3 trains and evaluates on the real GTSRB dataset at [gtsrb/Train](#).

Recorded Part 3 result (real GTSRB):

- Images loaded: 39,209
- Held-out test split: 7,842
- Epochs: 6, batch size: 32
- Accuracy: 0.9872 (7,742 / 7,842 correct)
- Saved model: [reports/artifacts/gtsrb_model.keras](#)

Setup and paths

The next cell sets shared paths, helper utilities, and quick sanity checks. It reads the canonical metrics file at [reports/evaluation_metrics.json](#) and uses figures in [reports/figures](#).

```
In [ ]: from pathlib import Path
import json
import os

import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import pandas as pd

PROJECT_ROOT = Path.cwd()
if not (PROJECT_ROOT / "src").exists() and (PROJECT_ROOT.parent / "src").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

REPORTS_DIR = PROJECT_ROOT / "reports"
FIG_DIR = REPORTS_DIR / "figures"
METRICS_PATH = REPORTS_DIR / "evaluation_metrics.json"
MODEL_PATH = REPORTS_DIR / "artifacts" / "gtsrb_model.keras"
GTSRB_DIR = PROJECT_ROOT / "gtsrb" / "Train"

def resolve_path(path_str):
    return PROJECT_ROOT / Path(path_str)

def show_image(path, title=None, figsize=(8, 5)):
    image = mpimg.imread(path)
```

```
plt.figure(figsize=figsize)
plt.imshow(image)
plt.axis("off")
if title:
    plt.title(title)
plt.show()

print("Project root:", PROJECT_ROOT)
print("GTSRB dir exists:", GTSRB_DIR.is_dir())
print("Metrics file:", METRICS_PATH)
print("Model artifact:", MODEL_PATH)
```

Part 1 - Flight Connections (BFS)

Implementation: [src/part1_search/flights.py](#)

Test cases (demo flight graph):

Case	Expected hops	Actual hops	Passed
Windhoek to Cairo	3	3	True
Johannesburg to Lagos	1	1	True
Nairobi to Nairobi	0	0	True
Cairo to Windhoek	None	None	True

Figure: [reports/figures/part1_route_lengths.png](#)

```
In [ ]: from src.evaluation.run_evaluation import run_part1

part1 = run_part1()
part1_results = pd.DataFrame(part1["results"])
part1_results[["case", "expected_hops", "actual_hops", "passed"]]

show_image(resolve_path(part1["figure"]), "Part 1 route lengths")
```

Part 2 - Hospital Shift Scheduler (CSP)

Implementation: [src/part2_optimization/run_scheduler.py](#) and [src/models/Csp.py](#)

Demo metrics:

- All 21 shifts assigned, no leave violations, no rest-rule violations.
- Max shifts per nurse: 5
- Fairness standard deviation: 1.6

Shift counts (demo):

Nurse	Shifts
Alice	5
Bob	5
Carol	5
David	5
Eve	1

Figures:

- [reports/figures/part2_shift_distribution.png](#)
- [reports/figures/part2_schedule_overview.png](#)

```
In [ ]: from src.evaluation.run_evaluation import run_part2

part2 = run_part2()
metrics = part2["metrics"]

pd.DataFrame([metrics])[
    ["complete", "fully_assigned", "passes_constraints", "max_shifts_per_nurse", "f
"].rename(columns={"max_shifts_per_nurse": "max_shifts"})

shift_counts = pd.DataFrame(
    [{"nurse": nurse, "shifts": count} for nurse, count in metrics["shift_counts"].
).sort_values("nurse")
shift_counts

for fig in part2["figures"]:
    title = f"Part 2 - {Path(fig).stem.replace('_', ' ').title()}"
    show_image(resolve_path(fig), title)
```

Part 3 - Traffic Sign Recognition (CNN)

Implementation: [src/train.py](#) and [src/models/Cnn.py](#)

Recorded evaluation (real GTSRB):

- Dataset: [gtsrb/Train](#)
- Images loaded: 39,209
- Held-out test split: 7,842
- Epochs: 6
- Batch size: 32
- Accuracy: 0.9872
- Correct predictions: 7,742 / 7,842
- Model artifact: [reports/artifacts/gtsrb_model.keras](#)

Figures:

- [reports/figures/confusion.png](#)
- [reports/figures/sample_predictions.png](#)
- [reports/figures/training_curve.png](#)

```
In [ ]: if not GTSRB_DIR.is_dir():
        raise FileNotFoundError("Expected GTSRB dataset at gtsrb/Train")

with METRICS_PATH.open("r", encoding="utf-8") as handle:
    metrics = json.load(handle)

part3 = metrics["part3"]
part3_summary = pd.DataFrame(
    [
        {
            "dataset": part3["dataset"],
            "total_images": part3["total_images"],
            "test_images": part3["test_images"],
            "epochs": part3["epochs"],
            "batch_size": part3["batch_size"],
            "accuracy": round(part3["accuracy"], 4),
            "correct_predictions": f"{part3['correct_predictions']} / {part3['test_
            "model_artifact": part3["model_artifact"],
        }
    ]
)
part3_summary

for fig in part3["figures"]:
    title = f"Part 3 - {Path(fig).stem.replace('_', ' ').title()}"
    show_image(resolve_path(fig), title)

# Optional rerun (will retrain):
# from src.evaluation.run_evaluation import run_part3
# os.environ["GTSRB_DIR"] = str(GTSRB_DIR)
# part3_run = run_part3()
# part3_run
```

Report and reference artifacts

The notebook aligns with the final report artifacts:

- Background and theory: [reports/background.md](#)
- Literature summary: [references/literature-summary.md](#)
- Results summary: [reports/results.md](#)
- Evaluation notes: [reports/evaluation_notes.md](#)
- Canonical metrics: [reports/evaluation_metrics.json](#)
- Slide storyline: [presentation/slide_draft.md](#)